

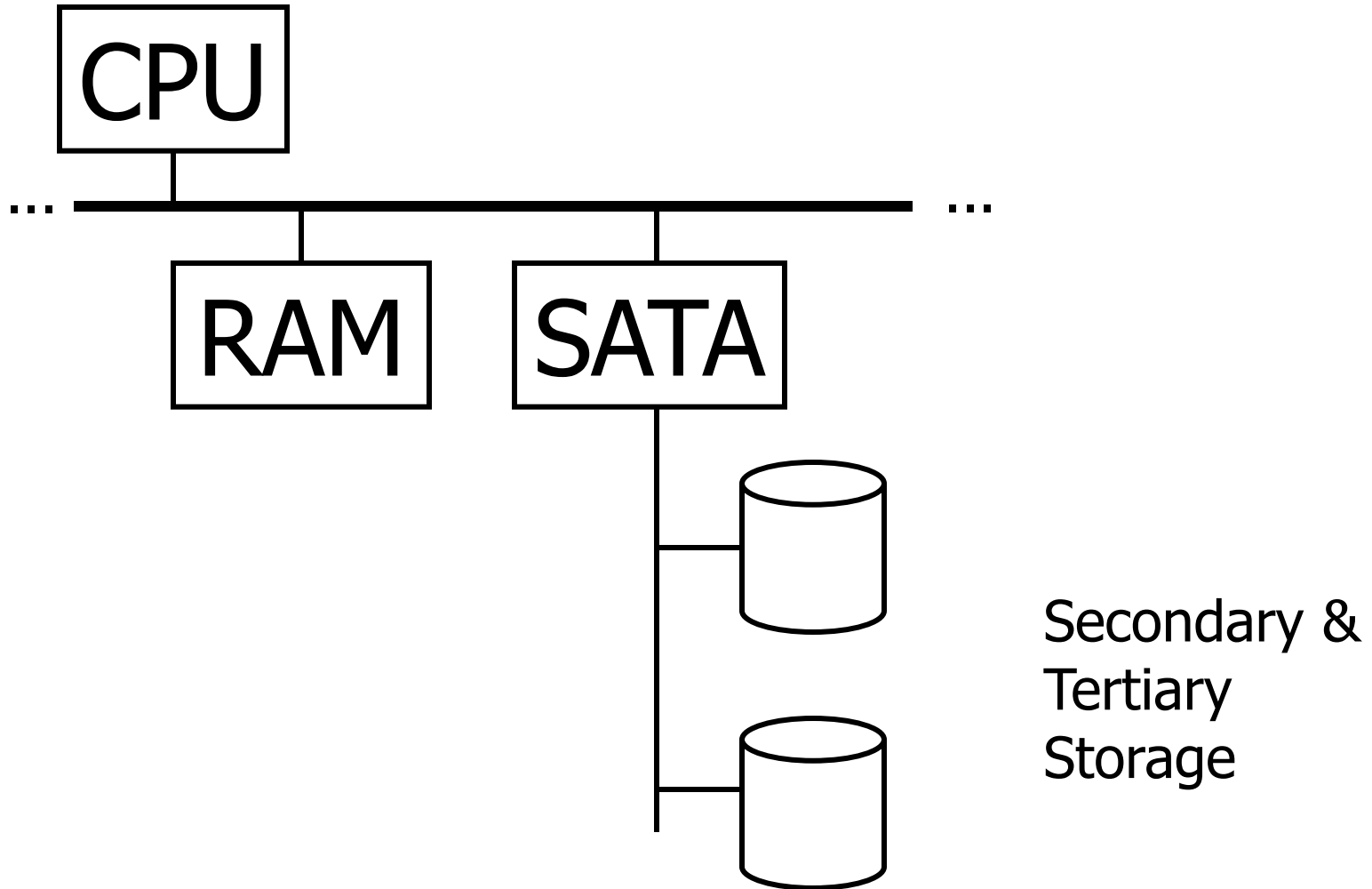
DM505 Database Design and Programming DM576 Database Systems

Panagiotis Tampakis

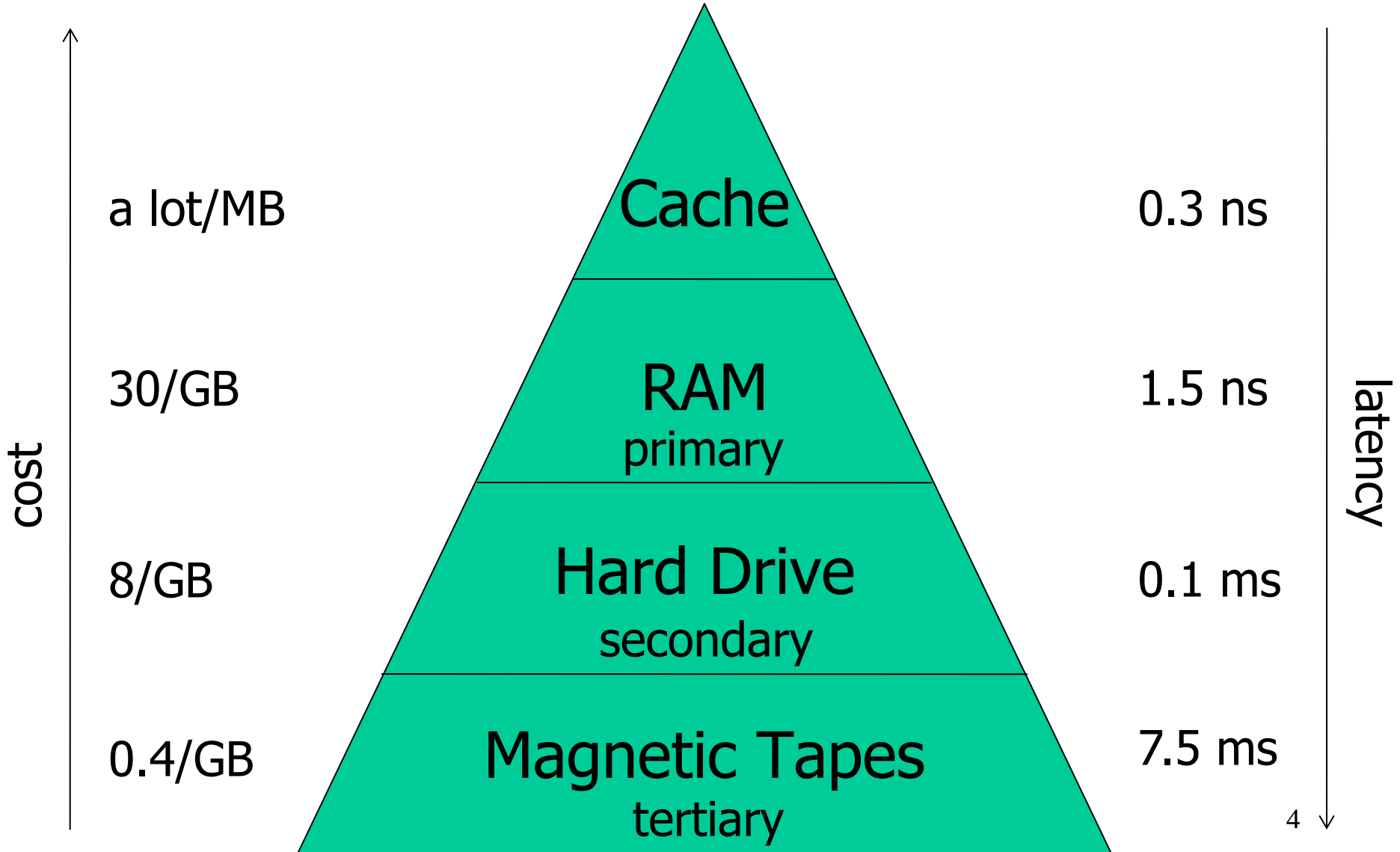
ptampakis@imada.sdu.dk

Data Storage

Computer System



The Memory Hierarchy

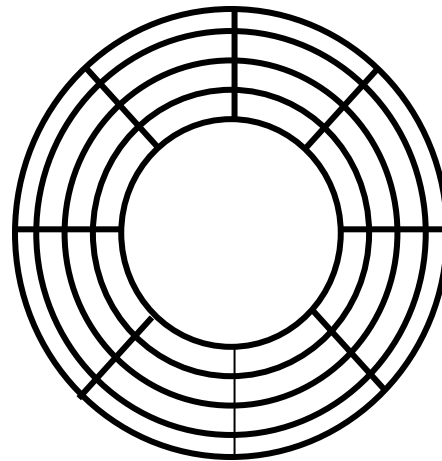
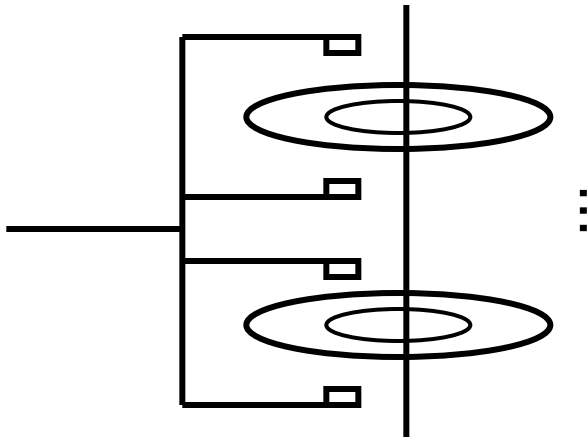


DBMS and Storage

- Databases typically too large to keep in primary storage
- Tables typically kept in secondary storage
- Large amounts of data that are only accessed infrequently are stored in tertiary storage (or even on tape robot)
- Indexes and current tables *cached* in primary storage

Harddisk

- N rotating magnetic platters
- $2 \times N$ heads for reading and writing
- track, cylinder, sector, gap

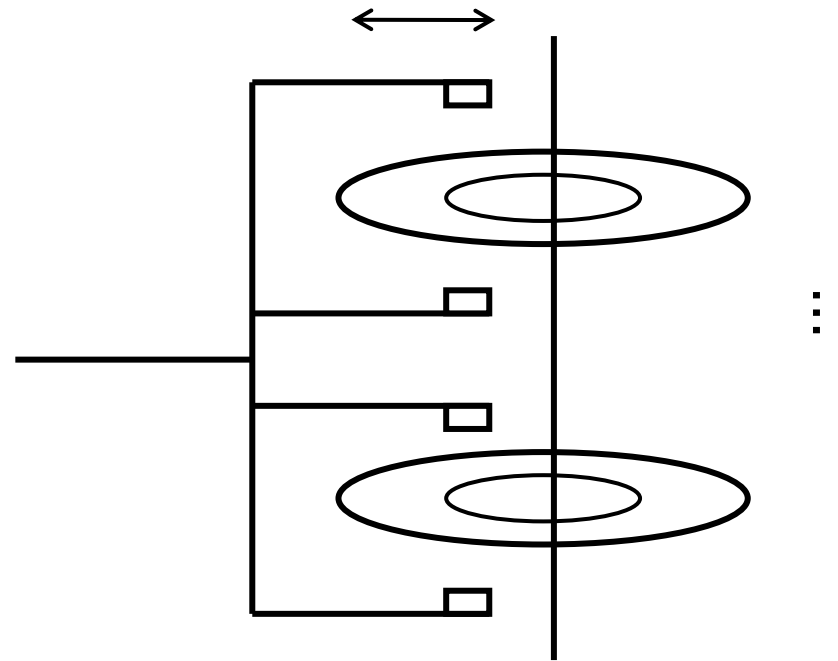


Harddisk Access

- **access time:** how long does it take to load a block from the harddisk?
- **seek time:** how long does it take to move the heads to the right cylinder?
- **rotational delay:** how long does it take until the head gets to the right sectors?
- **transfer time:** how long does it take to read the block?
- $\text{access} = \text{seek} + \text{rotational} + \text{transfer}$

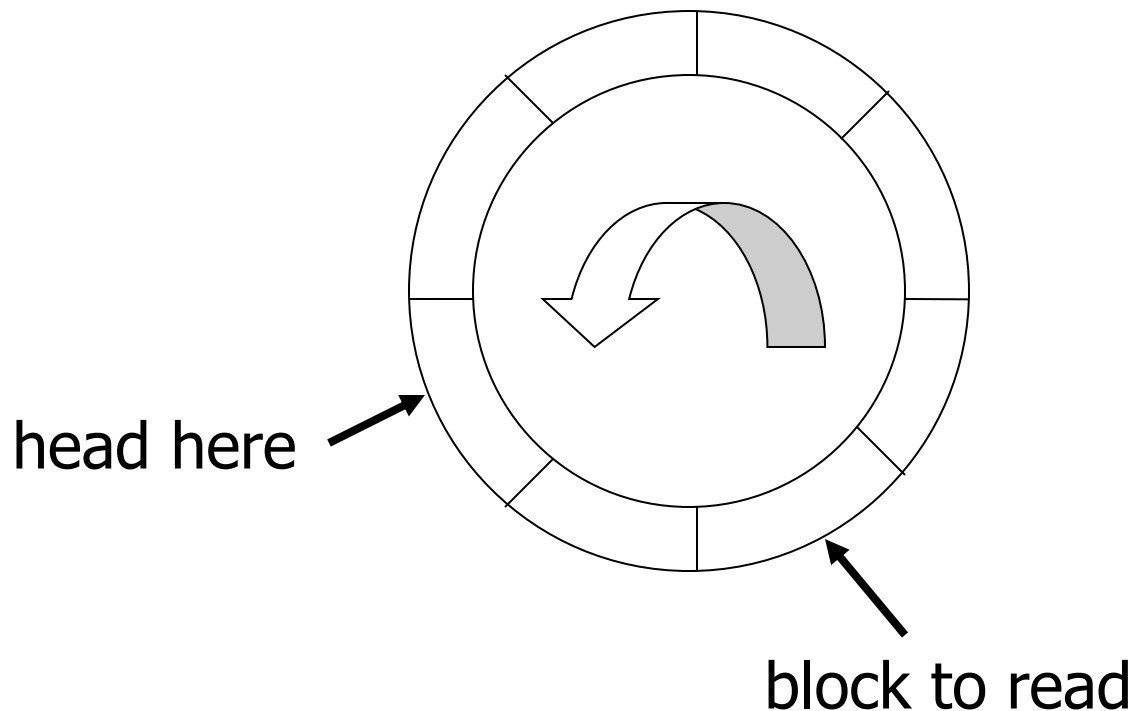
Seek Time

- average seek time =
1/3 time to move
head from outermost
to innermost cylinder
 - one millisecond to
start and stop, plus
one additional
millisecond for every
4000 cylinders



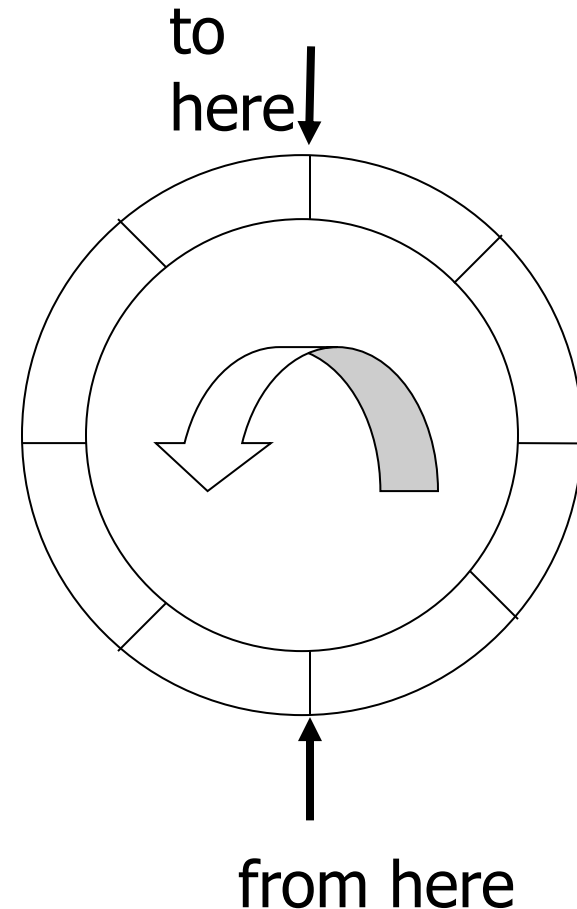
Rotational Delay

- average rotational delay = $\frac{1}{2}$ rotation



Transfer Time

- Transfer time of a block =
- $\frac{\phi^\circ}{360} \times t$
 - ϕ° is the total degrees of arc covered by sectors and gaps
 - t is the time for a full rotation



Access Time

- Typical harddisk:
 - Maximal seek time: 10 ms
 - Rotational speed: 7200 rpm
 - Block size: 4096 bytes
 - Sectors (512 bytes) per track: 1600 (average)
- Average access time: **7.51 ms**
 - Average seek time: 3.3 ms
 - Average rotational delay: $60/7200/2 = 4.17$ ms
 - Average transfer time: 0.04 ms

Random vs Sequential Access

- Random access of blocks:
 - $0.00751\text{s} * 256 \text{ blocks/Mbyte} = 1.92256 \text{ s/Mbyte}$
 - $1/1.92256 = 0.52 \text{ Mbyte/s}$
- Sequential access of blocks:
 $(60/7200) * ((512 * 1600)/4096) * 4096 \text{ byte} = 120/\text{s} * 200 * 4096 \text{ byte} = 94 \text{ Mbyte/s}$
- Performance of the DBMS dominated by number of random accesses

The I/O Model of Computation

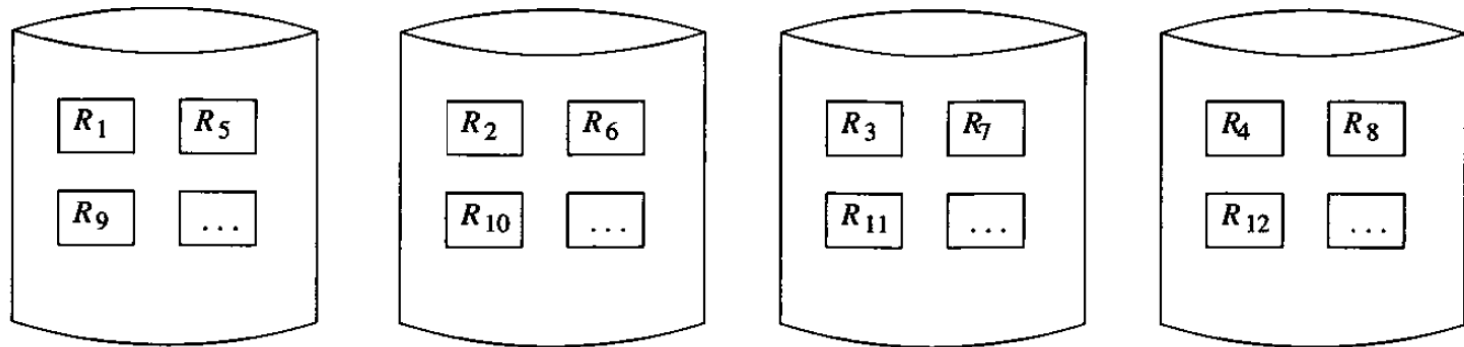
- DBMSs are I/O intensive
 - The time taken to perform a disk access is much larger than the time likely to be used manipulating that data in main memory.
 - Thus, the number of block accesses (Disk I/O's) is a good approximation to the time needed by the algorithm and should be minimized.

Accelerating Access

- Place blocks that are accessed together on the same cylinder.
 - we can often avoid seek time, and possibly rotational latency as well.
- Divide the data among several smaller disks rather than one large one.
 - Having more head assemblies that can go after blocks independently can increase the number of block accesses per unit time.

Accelerating Access

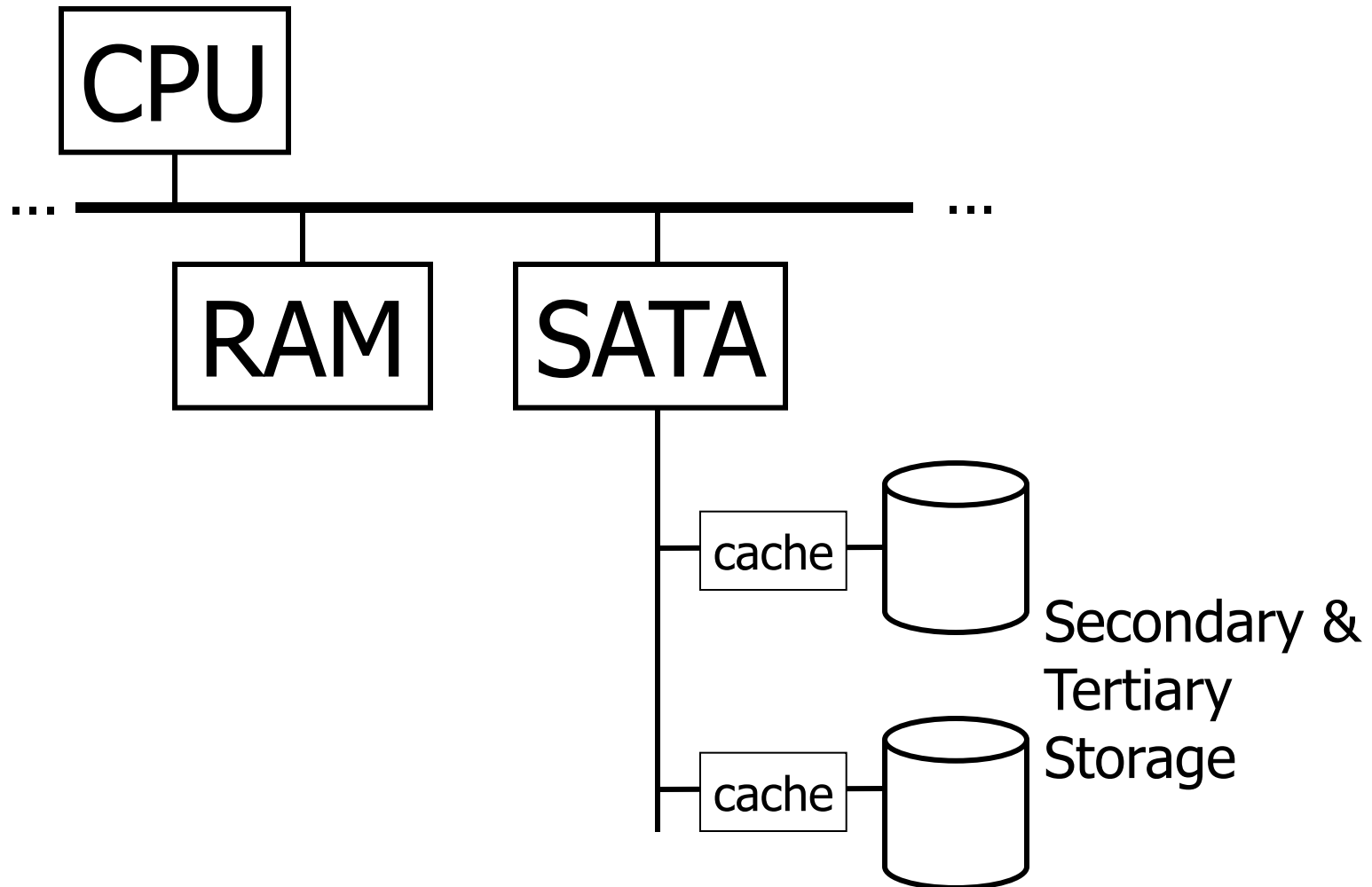
- “Mirror” a disk: making two or more copies of the data on different disks.
 - lets us access several blocks at once.



Accelerating Access

- Use a disk-scheduling algorithm (either in the operating system, the DBMS, or the disk controller)
 - select the order in which several requested blocks will be read or written.
- Prefetch blocks to main memory in anticipation of their later use.

On Disk Cache



Exercise

Exercise 13.3.1: Suppose we are scheduling I/O requests for a Megatron 747 disk, and the requests in Fig. 13.9 are made, with the head initially at track 32,000. At what time is each request serviced fully if:

- a) We use the elevator algorithm (it is permissible to start moving in either direction at first).
- b) We use first-come-first-served scheduling.

Seek Time: $1 + (\text{Tracks\#/}4000)$

Rotational Latency: 4.17

Transfer Time: 0.13

Cylinder of request	First time available
8000	0
24000	0
56000	0
16000	10
64000	20

Problems with Harddisks

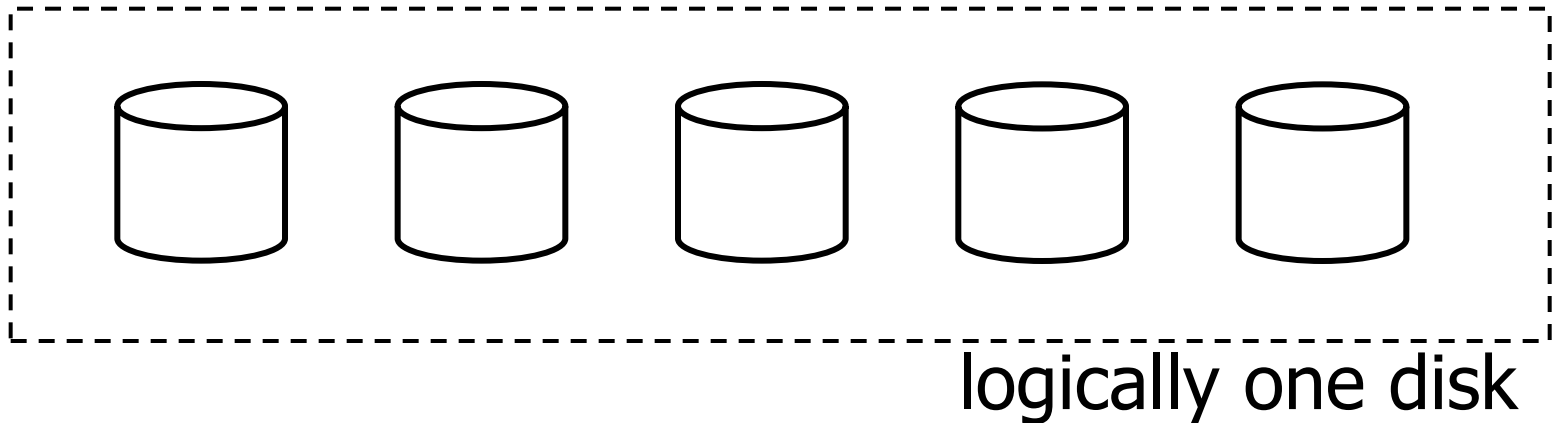
- Even with caches, harddisk remains bottleneck for DBMS performance
- Harddisks can fail:
 - Intermittent failure
 - Media decay
 - Write failure
 - Disk crash
- Handle intermittent failures by rereading the block in question

Detecting Read Failures

- Use checksums to detect failures
- Simplest form is parity bit:
 - 0 if number of ones in the block is even
 - 1 if number of ones in the block is odd
 - Detects all 1-bit failures
 - Detects 50% of many-bit failures
 - By using n bits, we can reduce the chance of missing an error to $1/2^n$

Disk Arrays

- Use more than one disk for higher reliability and/or performance
- RAID (Redundant Arrays of Independent Disks)



RAID 0

- Alternate blocks between two or more disks ("Striping")
- Increases performance both for writing and reading
- No increase in reliability

Disk	1	2
	0	1
	2	3
	4	5

← Storing blocks 0-5
in the first three
blocks of disk 1 & 2

RAID 1

- Duplicate blocks on two or more disks ("Mirroring")
- Increases performance for reading
- Increases reliability significantly

Disk	1	2
	0	0
	1	1
	2	2

← Storing blocks 0-2
in the first three
blocks of disk 1 & 2

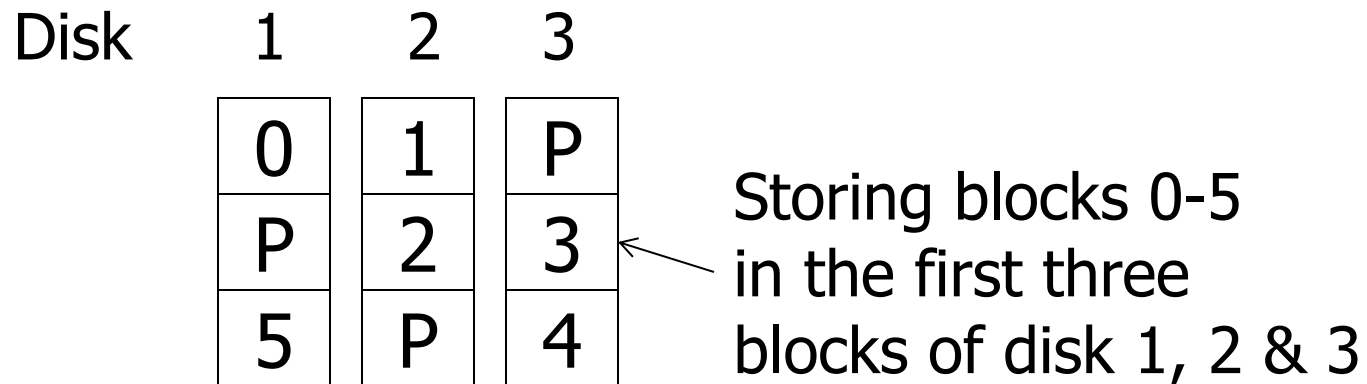
RAID 4

- Stripe blocks on $n+1$ disks where one disk stores parity information
- More performant when writing than RAID 1
- Increased reliability compared to RAID 0

Disk	1	2	3
	0	1	P
	2	3	P
	4	5	P

RAID 5

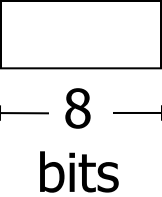
- Stripe blocks on $n+1$ disks where for each block, one disk stores parity information.
- More efficient than RAID 4.



RAID Capacity

- Assume disks with capacity 1 TByte
- RAID 0: N disks = N TByte
- RAID 1: N disks = 1 Tbyte
- RAID 4: N disks = $(N-1)$ TByte
- RAID 5: N disks = $(N-1)$ TByte

Storage of Values

- Basic unit of storage: Byte 
- Integer: 4 bytes

Example: 42 is

00000000 00000000 00000000 00101010

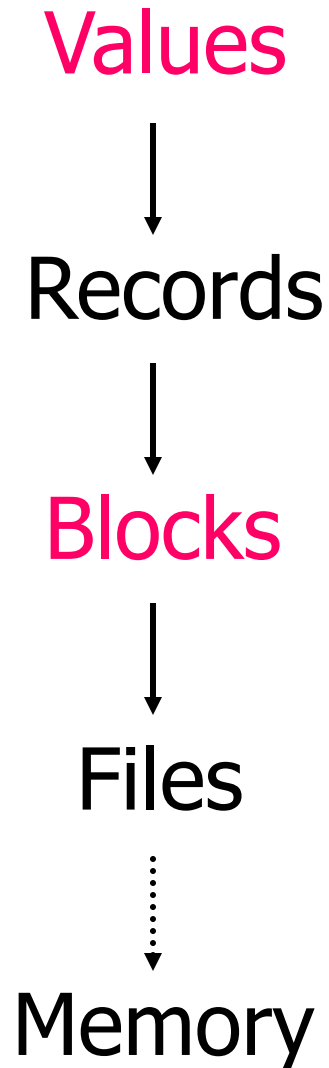
- Characters: ASCII, UTF8, ...
- Boolean: 00000000 and 11111111

Storage of Values

- Dates:
 - Days since January 1, 1900
 - DDMMYYYY (not DDMMYY)
- Time:
 - Seconds since midnight
 - HHMMSS
- Strings:
 - Null terminated
 - Length given

L	a	r	s	✕	
4	L	a	r	s	

DBMS Storage Overview



Record

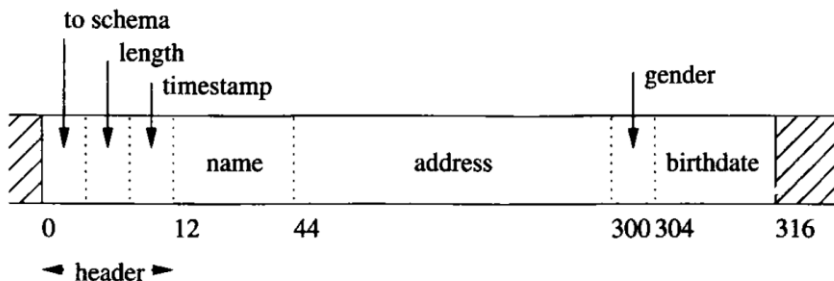
- Collection of related data items (called Fields)
- Typically used to store one tuple
- **Example:** Sells record consisting of
 - bar field
 - beer field
 - price field

Record Metadata

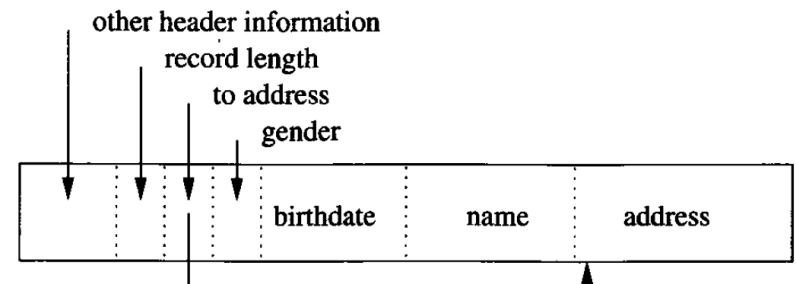
- For fixed-length records, schema contains the following information:
 - Number of fields
 - Type of each field
 - Order in record
- For variable-length records, every record contains this information in its header

Record Header

- Reserved part at the beginning of a record
- Typically contains:
 - Record type (which Schema?)
 - Record length (for skipping)
 - Time stamp (last access)



Fixed Length



Variable Length

Blocks

- Options for storing records in blocks:
 1. Separating records
 2. Spanned vs. unspanned
 3. Sequencing
 4. Indirection

1. Separating Records

Block



a. no need to separate - fixed size recs.

b. special marker

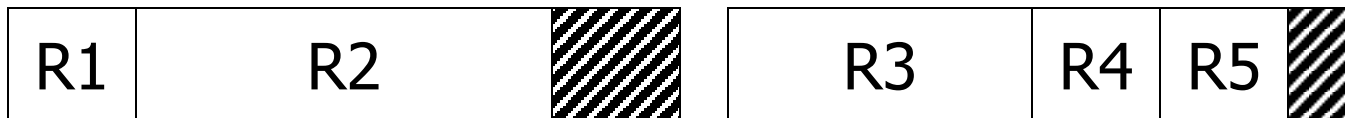
c. give record lengths (or offsets)

i. within each record

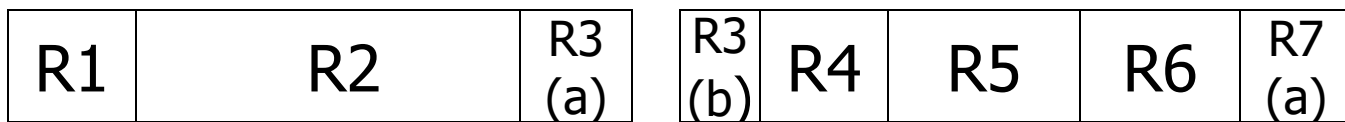
ii. in block header

2. Spanned vs Unspanned

- **Unspanned:** records must be in one block



- **Spanned:** one record in two or more blocks



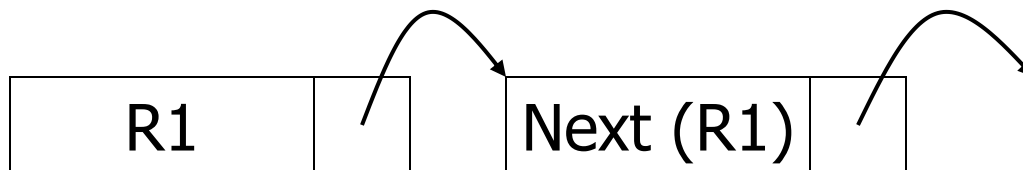
- Unspanned much simpler, but wastes space
- Spanned necessary if record size > block size

3. Sequencing

- Ordering records in a file (and in the blocks) by some key value
- Can be used for binary search
- Options:
 - a. Next record is physically contiguous

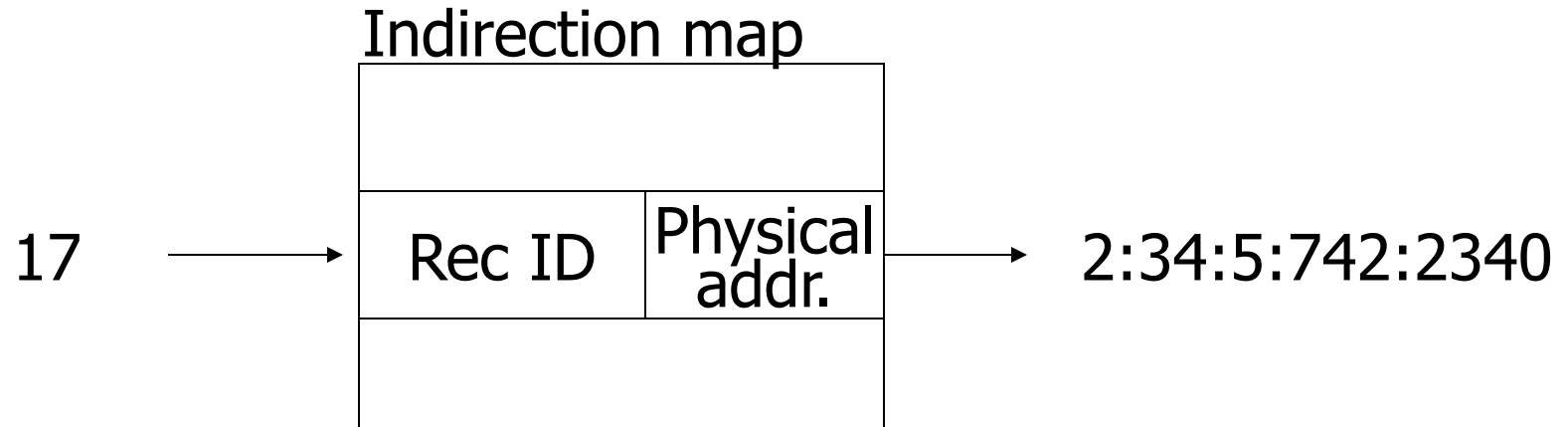


- b. Records are linked



4. Indirection

- How does one refer to records?
 - a. Physical address (disk id, cylinder, head, sector, offset in block)
 - b. Logical record ids and a mapping table



- Tradeoff between flexibility and cost

Modification of Records

How to handle the following operations on the record level?

1. Insertion
2. Deletion
3. Update

1. Insertion

- **Easy case:** records not in sequence
 - Insert new record at end of file
 - If records are fixed-length, insert new record in deleted slot
- **Difficult case:** records are sorted
 - Find position and slide following records
 - If records are sequenced by linking, insert overflow blocks

2. Deletion

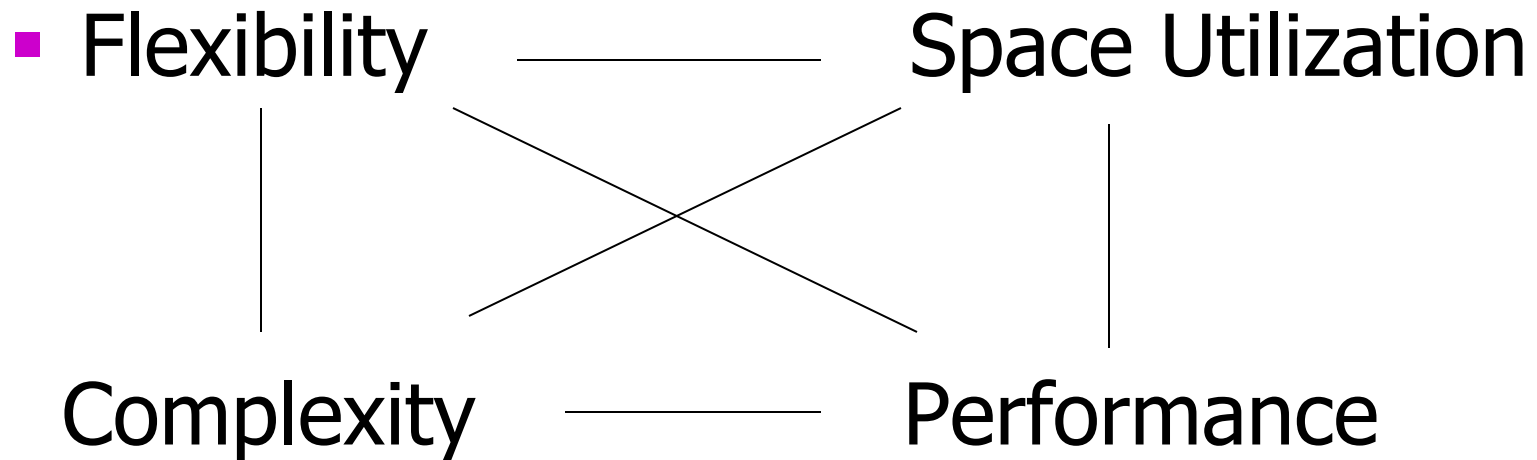
- a. Immediately reclaim space by shifting other records or removing overflows
- b. Mark deleted and list as free for re-use
 - Tradeoffs:
 - How expensive is immediate reclaim?
 - How much space is wasted?

3. Update

- If records are fixed-length and the order is not affected:
 - Fetch the record, modify it, write it back
- Otherwise:
 - Delete the old record
 - Insert the new record overwriting the tombstones from the deletion

Data Organizaton

- There are millions of ways to organize the data on disk



Summary 13

More things you should know:

- Memory Hierarchy
- Values, Records, Blocks, Files
- Storing and modifying records